

IA PARA DEVS

FASE 3 | AULA 01 -

INTRODUÇÃO AO LANGGRAPH

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	8
MERCADO, CASES E TENDÊNCIAS.....	13
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS	16

EMENDAS

O QUE VEM POR AÍ?

Você viu como usamos o LangChain para montar fluxos com LLMs, chamar um modelo, recuperar dados e montar uma resposta. Mas você já teve a sensação de que, conforme o fluxo cresce, tudo vira um emaranhado difícil de seguir?

E se houvesse uma forma mais clara, modular e flexível de organizar essas etapas? É aí que entra o LangGraph. Ao invés de seguir uma linha reta, ele permite implementar agentes de IA para conversar entre si, montando em uma estrutura flexível e poderosa um grafo de decisões. Podemos ter um ou múltiplos agentes e controlar o caminho que os dados percorrem. É uma nova forma de orquestrar a inteligência, pronta para lidar com sistemas mais complexos, sem perder o controle.

HANDS ON

Vamos montar seu primeiro grafo com LangGraph. A proposta é simples e divertida: prever o “clima da sua personalidade” com base em seu nome. Para isso, usaremos dois nós: um nó utilizando uma LLM da OpenAI, que retorna uma previsão simbólica de clima (Ensolarado, Nublado ou Chuvoso); e um nó Python, que interpreta esse clima e sugere uma atitude ou reflexão para o seu dia.

Esse hands on vai te mostrar como: criar nós e conectá-los com arestas, usar o estado compartilhado para transmitir informações, aplicar lógica condicional de forma confiável e visualizar a estrutura do seu grafo.

Observação: vamos usar python-dotenv para carregar sua chave da OpenAI com segurança a partir de um arquivo .env.

ETAPA 0 – INSTALAÇÃO

Execute este comando no terminal para garantir que tudo está instalado:

```
# INSTALAÇÃO - TERMINAL
> pip install langgraph langchain langchain-openai python-dotenv grandalf
```

Código-fonte 1 – Instalação Terminal
Fonte: elaborado pelo autor (2025)

Crie um arquivo .env na raiz do projeto com sua chave da OpenAI:

```
OPENAI_API_KEY=sk-...
```

Código-fonte 2 – Chave da OpenAI
Fonte: elaborado pelo autor (2025)

ETAPA 1 – PREPARANDO O AMBIENTE E CARREGANDO A CHAVE DA OPENAI

Carregamos a chave da API usando a biblioteca dotenv:

```
import os
from dotenv import load_dotenv
from typing import TypedDict
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
```

```
from langgraph.graph import StateGraph, END

load_dotenv()
os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY")
```

Código-fonte 3 – Carregamento da Chave da OpenAI usando a biblioteca dotenv
Fonte: elaborado pelo autor (2025)

ETAPA 2 – DEFININDO O ESQUEMA DE ESTADO

No LangGraph, o estado compartilhado entre os nós pode ser representado por um dicionário “tipado”. Vamos definir um estado simples que contém o nome da pessoa, o clima retornado e a mensagem_final:

```
class EstadoClima(TypedDict):
    nome: str
    clima: str
    mensagem_final: str
```

Código-fonte 4 – Definição de estado simples
Fonte: elaborado pelo autor (2025)

ETAPA 3 – CRIANDO O PRIMEIRO NÓ: PREVISÃO COM LLM

Este nó usa um prompt fixo para pedir ao LLM que escolha um dos três tipos de clima. Esse prompt precisa ser bem controlado, para facilitar o uso condicional depois.

```
prompt = PromptTemplate.from_template(
    "Com base no nome {nome}, escolha o clima da personalidade da pessoa hoje. "
    "Responda apenas com uma das três palavras: Ensolarado, Nublado ou Chuvoso."
)

llm = ChatOpenAI(model="gpt-3.5-turbo")

def prever_clima(state: dict) -> dict:
    nome = state["nome"]
    full_prompt = prompt.format_prompt(nome=nome).to_string()
    resposta = llm.invoke(full_prompt)
    return {"clima": resposta.content.strip() }
```

Código-fonte 5 – Criação do primeiro nó para escolha do tipo de clima
Fonte: elaborado pelo autor (2025)

ETAPA 4 – CRIANDO O SEGUNDO NÓ: LÓGICA CONDICIONAL

Agora que temos o clima, vamos interpretar esse valor e retornar uma mensagem divertida e apropriada para cada caso.

```
def interpretar_clima(state: dict) -> dict:
    clima = state["clima"].strip().lower()

    if clima == "ensolarado":
        mensagem = "Dia ideal para brilhar – mostre sua energia ao mundo!"
    elif clima == "nublado":
        mensagem = "Dia introspectivo. Respire fundo e vá com calma."
    elif clima == "chuvoso":
        mensagem = "Dia de se recolher, mas também de nutrir ideias criativas."
    else:
        mensagem = "Clima indefinido... talvez você seja uma tempestade?"

    return {"mensagem_final": mensagem}
```

Código-fonte 6 – Criação do segundo nó

Fonte: elaborado pelo autor (2025)

Note como usamos **condicionais explícitas**, baseadas no valor gerado pelo nó anterior.

ETAPA 5 - CONSTRUINDO O GRAFO

Agora é hora de conectar os nós em um grafo funcional, com ponto de entrada e fim definidos.

```
workflow = StateGraph(EstadoClima)

workflow.add_node("prever_clima", prever_clima)
workflow.add_node("interpretar", interpretar_clima)

workflow.set_entry_point("prever_clima")
workflow.add_edge("prever_clima", "interpretar")
workflow.set_finish_point("interpretar")

app = workflow.compile()
```

Código-fonte 7 – Conectando os nós em um grafo funcional

Fonte: elaborado pelo autor (2025)

ETAPA 6 – VISUALIZANDO A ESTRUTURA DO GRAFO

Vamos imprimir a representação ASCII do grafo, útil para entender o fluxo de execução:

```
print("Estrutura do grafo:")
print(app.get_graph().draw_ascii())
```

Código-fonte 8 – Representação ASCII do grafo
Fonte: elaborado pelo autor (2025)

A saída será algo assim:

```
Estrutura do grafo:
+-----+
| __start__ |
+-----+
*
*
*
+-----+
| prever_clima |
+-----+
*
*
*
+-----+
| interpretar |
+-----+
*
*
*
+-----+
| __end__ |
+-----+

Previsão: Ensolarado
Mensagem: Dia ideal para brilhar – mostre sua energia ao mundo!
```

Figura 1 – Saída da representação ASCII do grafo
Fonte: elaborado pelo autor (2025)

ETAPA 7 – EXECUTANDO O GRAFO

Agora, vamos invocar o grafo passando o nome como entrada. O resultado inclui tanto a previsão quanto a mensagem final:

```
entrada = {"nome": "José"}
resultado = app.invoke(entrada)

print("\nPrevisão:", resultado.get("clima", "<sem retorno>"))
print("Mensagem:", resultado.get("mensagem_final", "<sem retorno>"))
```

Código-fonte 9 – Executando o grafo
Fonte: elaborado pelo autor (2025)

SAIBA MAIS

O LangGraph é uma biblioteca pensada para resolver um problema real: como organizar fluxos de execução envolvendo modelos de linguagem de forma mais clara, modular e controlável. Em vez de scripts lineares e difíceis de manter, o LangGraph propõe representar a lógica da aplicação como um grafo — ou seja, um conjunto de nós (etapas) conectados por arestas (transições). Cada nó representa uma ação: chamar um LLM, executar uma função, tomar uma decisão, recuperar um dado, entre outros.

Essa representação permite acompanhar exatamente o que está acontecendo em cada etapa do processo, isolar falhas, redirecionar fluxos com base em condições e reutilizar componentes com facilidade. Portanto, o LangGraph é uma maneira mais estruturada de pensar o próprio fluxo da aplicação, desde o seu conceito até sua implementação.

A EVOLUÇÃO DO LANGCHAIN AO LANGGRAPH

O LangChain foi uma das primeiras bibliotecas a ajudar pessoas desenvolvedoras a combinar modelos de linguagem com ferramentas externas, como mecanismos de busca, bancos vetoriais ou APIs. O modelo clássico do LangChain é linear: você conecta etapas em sequência, passando informações de uma para outra. Isso funciona para fluxos simples, mas rapidamente se torna um problema à medida que a lógica cresce, exigindo ramificações ou dependendo de verificações intermediárias.

O LangGraph surgiu como uma extensão natural dessa necessidade. Ele ainda usa os mesmos componentes fundamentais do LangChain, mas os organiza em grafos, sendo uma arquitetura que permite múltiplos caminhos possíveis, decisões condicionais, loops, paralelismo e muito mais. O que antes era um pipeline rígido e difícil de escalar se transforma em um sistema visualmente compreensível e modular.

POR QUE USAR GRAFOS?

Sistemas baseados em grafos são especialmente úteis quando precisamos lidar com fluxos que não são completamente previsíveis ou lineares. Por exemplo, imagine um assistente de viagem. Ao receber uma solicitação, ele pode:

- Identificar se a pessoa quer ajuda com passagens, hospedagem ou roteiros turísticos;
- Consultar um banco de dados ou API específica para cada caso;
- Gerar uma resposta baseada em diferentes documentos;
- Verificar se a resposta está adequada ou se deve escalar para um atendente humano.

Cada uma dessas ações pode ser representada como um nó e o caminho que a informação percorre no grafo vai depender da intenção da pergunta, da confiança da resposta ou de regras de negócios. Com LangGraph, é possível desenhar esse fluxo de forma clara, separando responsabilidades e facilitando testes, ajustes e expansões.

COMPONENTES DO LANGCHAIN REUTILIZADOS NO LANGGRAPH

Um dos pontos fortes do LangGraph é que ele não exige abandonar o que você já aprendeu com LangChain. Os principais blocos continuam valendo, só que agora encaixados dentro de nós que formam o grafo:

- **PromptTemplate:** continua sendo o meio de estruturar a entrada para o modelo de linguagem. A diferença é que o preenchimento dos valores agora pode vir diretamente do fluxo de dados que percorre o grafo.
- **LLMChain:** em vez de ser um pipeline isolado, ele agora pode ser usado dentro de um nó do LangGraph. Isso dá mais controle sobre quando e como ele é executado.
- **VectorStore:** o armazenamento e recuperação de vetores (como embeddings de contexto) se tornam nós reutilizáveis, que podem ser invocados por múltiplos caminhos no grafo.

- **Ferramentas e memória:** também podem ser organizadas em nós, garantindo um uso mais claro, com escopo bem definido e fácil de depurar.

Essa estrutura modular facilita a reutilização e permite isolar partes da aplicação, por exemplo, testando apenas o nó de busca vetorial ou modificando um prompt, sem interferir no restante da lógica.

AGENTES DE IA COMO NÓS

O principal benefício do LangGraph é a forma como ele libera o potencial dos agentes de IA. Em vez de um agente monolítico, com várias funções amarradas num só ponto, o LangGraph permite que cada etapa do raciocínio e da ação seja tratada como um agente especializado. Isso abre caminho para fluxos cooperativos, onde diferentes nós executam diferentes papéis — e o sistema toma decisões sobre qual nó ativar a seguir.

Cada agente pode ser uma combinação de LLM, ferramentas e lógica de decisão. Como tudo está representado em um grafo, é possível adicionar verificações, caminhos alternativos e ciclos sem tornar o código caótico. O resultado é um sistema mais robusto, testável e fácil de evoluir, pronto para lidar com aplicações reais, como assistentes de atendimento, classificadores inteligentes, geradores de conteúdo etc.

Tabela 1 – Diferenças entre LangChain e LangGraph

Conceito	LangChain	LangGraph
Estrutura	Sequencial (pipeline)	Gráfico (nós e arestas)
Modularidade	Baixa (tudo acoplado)	Alta (cada agente é um nó)
Reutilização de etapas	Limitada	Facilitada
Fluxos condicionais	Complexos, exige lógica manual	Naturais, com arestas condicionais
Visualização do fluxo	Mais difícil	Intuitiva
Escalabilidade (equipes, testes)	Limitada	Alta

Fonte: elaborado pelo autor (2025)

O ponto é: **o LangGraph não substitui o LangChain**, mas se torna um apoiodele. Tudo que você usou no LangChain (PromptTemplate, LLMChain, VectorStore) pode ser “encaixado” em nós do grafo.

COMO FUNCIONA NA PRÁTICA?

O ciclo básico de um grafo LangGraph é:

- Você define um estado inicial (por exemplo: {"pergunta": "Qual a capital da França?"});
- O grafo identifica quais nós podem ser executados com base nesse estado;
- Cada nó executa sua função (ex.: chamar um LLM, consultar um banco, validar algo);
- O resultado de cada nó é adicionado ao estado;
- Novos nós são liberados conforme suas dependências são satisfeitas;
- O fluxo continua até chegar em um ponto de parada (END).

Esse modelo é bastante útil em várias situações. Ele é ideal quando você precisa processar diferentes caminhos de execução, dependendo do conteúdo da entrada ou do nível de confiança da resposta gerada. Também é vantajoso quando deseja testar partes específicas do seu sistema de forma isolada, sem ter que executar todo o fluxo completo.

Além disso, ele permite especializar diferentes agentes em tarefas distintas — como um agente focado em busca de informações, outro dedicado à escrita e um terceiro responsável pela revisão. Por fim, se houver nós independentes, o LangGraph possibilita que esses trechos sejam executados em paralelo, acelerando a performance geral do sistema.

UM EXEMPLO RÁPIDO (E REAL)

Vamos retornar ao nosso exemplo de um assistente de viagem inteligente. O grafo pode ter um nó que recebe a solicitação do usuário com dados como destino desejado ou perfil de viagem. Em seguida, um segundo nó classifica a intenção da viagem (por exemplo, lazer, negócios ou aventura). Com base nisso, diferentes nós de busca são acionados: um para pacotes turísticos, outro para voos corporativos e outro para experiências ao ar livre.

Depois, um nó gera uma sugestão personalizada com base nas informações recuperadas. Um nó adicional pode verificar se a proposta atende a critérios como orçamento ou restrições do usuário. Por fim, o último nó envia a recomendação final ou encaminha para um agente humano especializado. Cada uma dessas etapas é tratada como um agente separado e o fluxo de execução varia conforme o perfil do viajante. Com LangChain, isso exigiria lógica manual espalhada e difícil de manter. Com LangGraph, você tem uma visão clara, modular e adaptável de todo o processo.

O QUE VOCÊ GANHA COM ISSO?

Com o uso de grafos, você ganha mais clareza ao visualizar o fluxo completo de execução, do início ao fim, etambém passa a ter mais controle, já que é possível inspecionar qualquer nó de forma isolada. A escalabilidade aumenta, pois se torna simples trocar o modelo de um agente, testar novos prompts ou integrar dados adicionais. Por fim, você conquista mais confiança no seu sistema, já que a estrutura em grafo facilita o processo de depurar, validar e evoluir a aplicação ao longo do tempo.

O LangGraph não é uma “nova moda”. É um reflexo natural do que acontece quando os projetos com LLMs crescem e precisam de mais controle, clareza e especialização. Ele não substitui o que você já sabe, apenas amplia. A verdadeira força do LangGraph não está apenas em sua arquitetura em grafo, mas na forma como obriga times de desenvolvimento a pensarem em modularidade, responsabilidade de componentes e testes isolados — princípios fundamentais em sistemas escaláveis baseados em IA.

MERCADO, CASES E TENDÊNCIAS

O LangGraph tem ganhado destaque como uma alternativa estruturada para orquestrar agentes de IA em fluxos mais complexos. Empresas de diversos setores estão adotando essa abordagem para construir sistemas mais modulares, escaláveis e fáceis de manter.

TENDÊNCIAS EMERGENTES

O uso de agentes de IA está em ascensão. Segundo o relatório “[State of AI Agents](#)” da LangChain, 51% das empresas já utilizam agentes em produção e 78% planejam implementá-los em breve. As principais aplicações incluem pesquisa e sumarização (58%), produtividade pessoal (53,5%) e atendimento ao cliente (45,8%).

Além disso, o **LangGraph Platform** foi lançado para facilitar o gerenciamento de agentes com estado persistente em larga escala. A plataforma oferece recursos como implantação com um clique, visualização de fluxos e escalabilidade horizontal, sendo adotada por quase 400 empresas desde seu lançamento beta. Veja mais [aqui](#)

APLICAÇÕES INOVADORAS

Empresas estão explorando o LangGraph para criar sistemas de análise de mercado, agentes de suporte ao cliente e ferramentas de pesquisa automatizada. Por exemplo, um projeto de análise de concorrência utiliza agentes especializados para coletar dados de sites de concorrentes e gerar insights estratégicos, automatizando tarefas que antes exigiam horas de trabalho manual.

- [Documentação Oficial do LangGraph](#).
- **Estudos de Caso:** [LangChain](#).

O LangGraph tem ganhado espaço como uma alternativa estruturada para orquestrar agentes de IA em fluxos mais complexos. Sua proposta facilita o desenvolvimento de sistemas mais organizados e modulares, o que tende a beneficiar aplicações que exigem controle maior sobre o comportamento dos

modelos em produção. Com a adoção crescente em projetos reais, o uso de grafos deve se consolidar como uma prática viável para lidar com tarefas que envolvem múltiplas etapas e decisões condicionais.



O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você conheceu o LangGraph e entendeu como ele amplia os conceitos do LangChain ao introduzir grafos de agentes de IA. Viu que, em vez de seguir fluxos lineares, agora é possível organizar tarefas em nós independentes e conectá-los com lógica condicional, deixando seu sistema mais modular, claro e escalável.

Apreendeu também como funcionam os principais componentes — nós, arestas e estado compartilhado — e colocou isso em prática com um hands on simples. Essa base será essencial para os próximos passos, em que os nossos fluxos ganharão mais inteligência e ramificações.

REFERÊNCIAS

BLOCKCHAIN NEWS. **LangGraph Platform Launches for Managing Stateful Agents at Scale**. Disponível em: <https://blockchain.news/news/langgraph-platform-launches-managing-stateful-agents>. Acesso em: 01 ago. 2025.

LANGGRAPH. **Repositório oficial no GitHub**. Disponível em: <https://github.com/langchain-ai/langgraph>. Acesso em: 01 ago. 2025.

LANGCHAIN. **State of AI Agents Report 2024**. Disponível em: <https://www.langchain.com/stateofaiagents>. Acesso em: 01 ago. 2025.

LANGCHAIN. **Top 5 LangGraph Agents in Production (2024)**. Blog LangChain. Disponível em: <https://blog.langchain.dev/top-5-langgraph-agents-in-production-2024>. Acesso em: 01 ago. 2025.

JALAJA GR. **LangGraph AI Agents Automate Competitor Analysis**. Medium. Disponível em: <https://medium.com/@jalajagr/langgraph-ai-agents-automate-competitor-analysis-a-step-by-step-guide-fbce1f956a3b>. Acesso em: 01 ago. 2025.

PALAVRAS-CHAVE

LangGraph. Agentes de IA. Grafos de Decisão.

EMEND



POSTECH